Generalized Additive Models and beyond with P-splines

Chapter 5

Dae-Jin Lee
BCAM - Basque Center for Applied Mathematics
XV Summer School MESIO, 2022

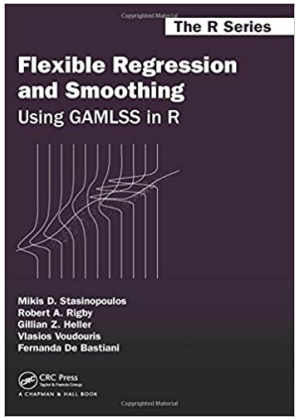
Barcelona, July 2022

5. Beyond GAMs

GAMLSS: Generalized Additive Models for Location, Shape and Scale

See https://www.gamlss.com/

GAMLSS are univariate distributional regression models, where all the parameters of the assumed distribution for the response can be modelled as additive functions of the explanatory variables.



- Several `R` packages `gamlss`, `gamlss.add`, `gamlss.dist` in CRAN
- For beginners and practitioners
- Reproducible code

What distributions can be used within GAMLSS?

GAMLSS provide over 100 continuous, discrete and mixed distributions for modelling the response variable. Truncated, censored, log and logit transformed and finite mixture versions of these distributions can be also used.

What additive terms can be used within GAMLSS?

P-splines, Cubic splines, loess smoothing, ridge regression, lasso regression, simple random effects and varying coefficient models are some of the additive functions provided in the implementation.

Appropriate interface is also provided so GAMLSS models can be used in combination with smoothers from the `gam()` function (of package `mgcv`), the neural network function `nnet()` (of package `nnet`) and decision trees (of package `rpart`).

What is a GAMLSS?

GAMLSS are distributional based semi-parametric regression type models (they are a generalization of GLM's and GAMs).

- **regression type:** we have many explanatory variables X and one response variable y and we believe that X explains y.
- **distributional:** a full parametric distribution assumption is made for the response variable.
- **semi-parametric:** All the parameters of the distribution can be modelled as function of the explanatory variables. Those functions can be parametric or non-parametric smoothing functions.

GAMLSS related R packages

- `gamlss` the original package
- `gamlss.dist` all gamlss.family distributions
- `gamlss.data` different sets of data
- `gamlss.add` for extra additive terms
- `gamlss.cens` for censored (left, right or interval) response variables
- `gamlss.demo` demos for distributions and smoothing
- `gamlss.inf` models for inflated 0 to 1 or for adjusted 0 to ∞
- `gamlss.nl` non-linear term fitting
- `gamlss.tr` generating truncated distributions
- `gamlss.mx` finite mixtures distributions and random effects
- `gamlss.spatial` for spatial models [GMRF models]
- `gamlss.util` for extra utilities

GAMLSS model formulation

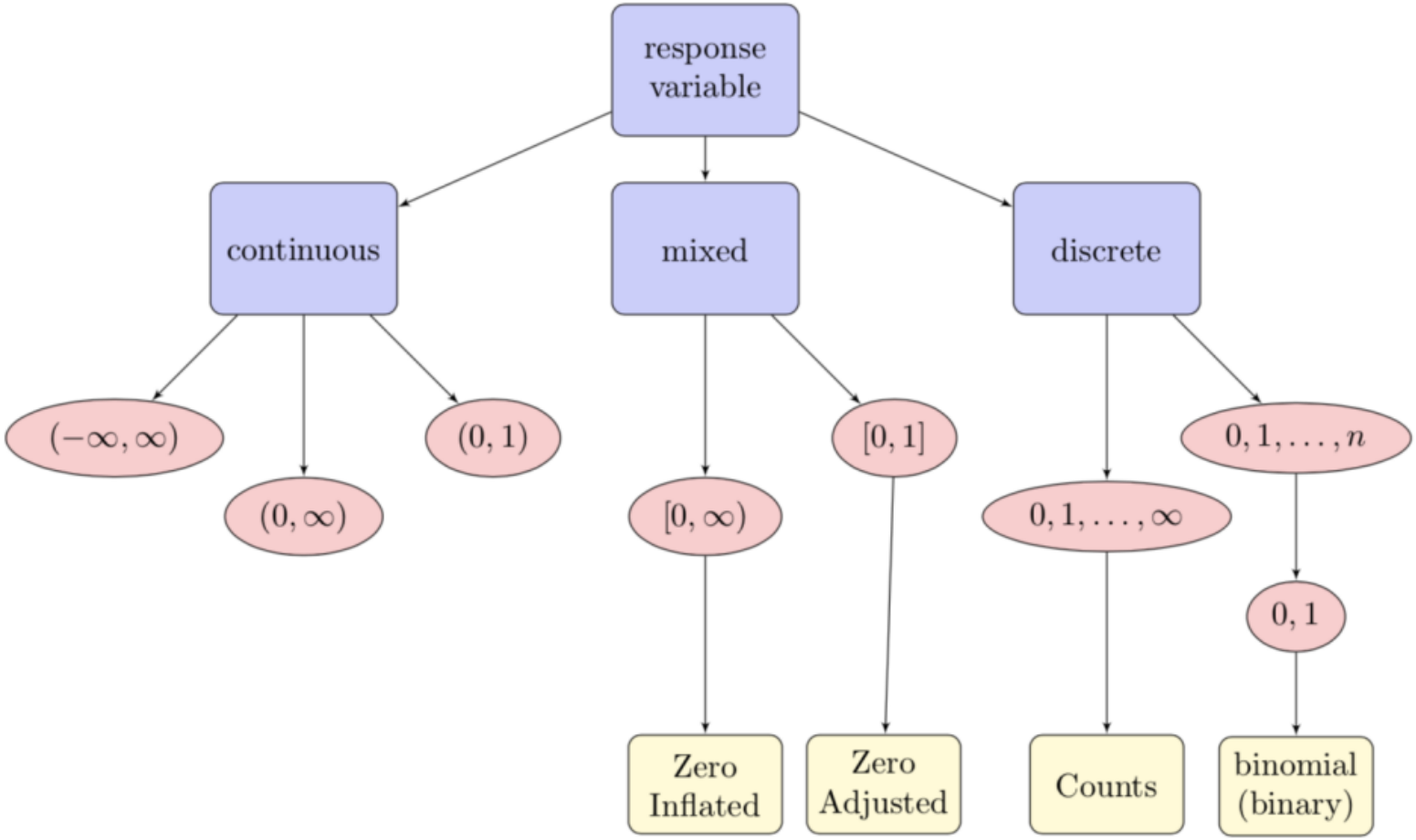
$$y \sim \text{Dist}(\mu, \sigma, \nu, \tau)$$

- $g_1(\mu) = X_1\beta_1 + f(x_{11}) + ...$
- $g_2(\sigma) = X_2\beta_2 + f(x_{21}) + ...$
- $g_3(\nu) = X_3\beta_3 + f(x_{31}) + ...$
- $g_4(\tau) = X_4\beta_4 + f(x_{41}) + ...$

Questions:

- What type of distribution is appropriate for the response variable?
- Which explanatory variable to select for each parameter?
- How those variable effect the distribution of the response?
- What link function shall we use?

Response variables



Different types of distributions

```
library(gamlss.demo)

demo.GA()
demo.PO()
demo.BI()
demo.BE()
demo.BEINF()
```

GAMs with `gamlss`

The library `gamlss` has several additive terms possibilities:

- P-splines (`pb`)
- Cubic splines (`cs`)
- Penalized Varying coefficient models (`pvc`)
- Cyclic P-splines (`pbc`)
- ...

gamlss.add

The `library(gamlss.add)` includes:

The `ga()` and `ba()` functions are a additive functions to be used within GAMLSS models.

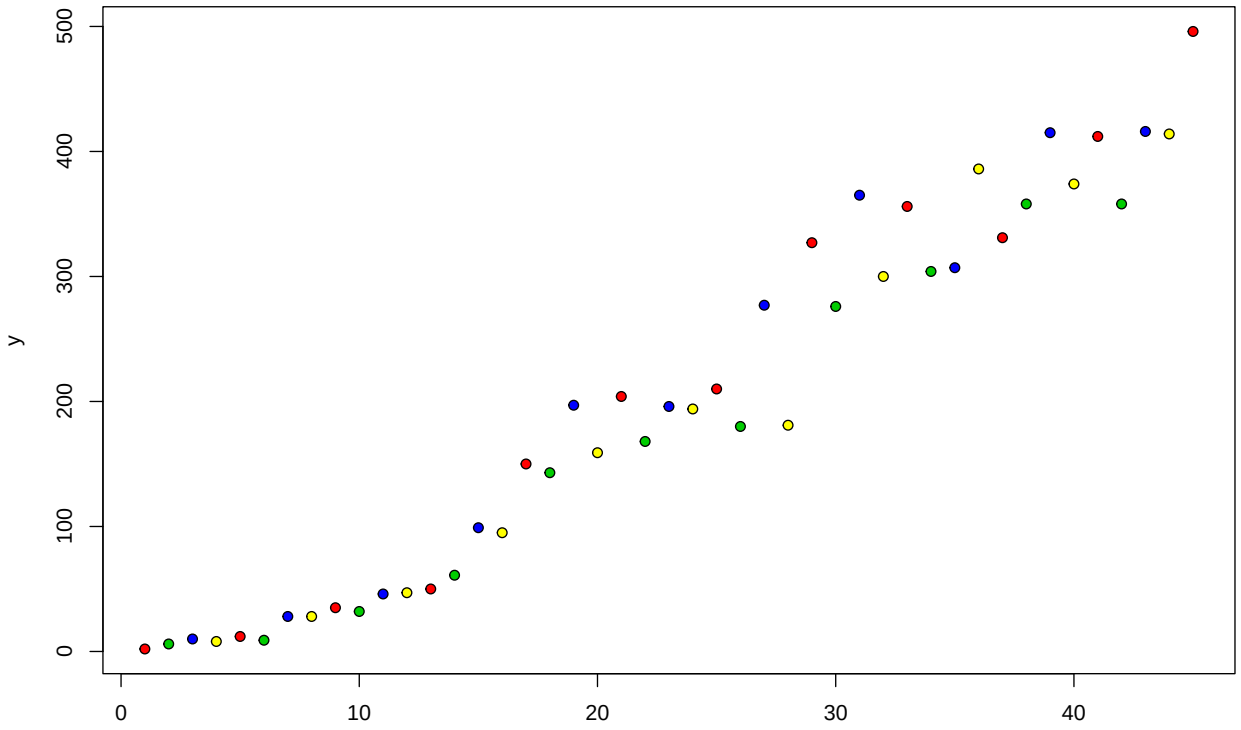
They are interfaces for the `gam()` and the `bam()` functions of package `mgcv` of Simon Wood.

The functions `gam()` and the `bam()` allows the user to use all the available smoothers of the package `mcgv()` within gamlss.

The great advantage of course come from fitting models outside the exponential family.

aids data

```
library(gamlss)
library(gamlss.add)
data(aids)
?aids

attach(aids)
plot(x,y,pch=21,bg=c("red","green3","blue","yellow")[unclass(qrt)])
```



```r
# fitting a smoothing cubic spline with 7 degrees of freedom
# plus the a quarterly  effect
aids1<-gamlss(y~ps(x,df=7)+qrt,data=aids,family=PO) # fix df's
```

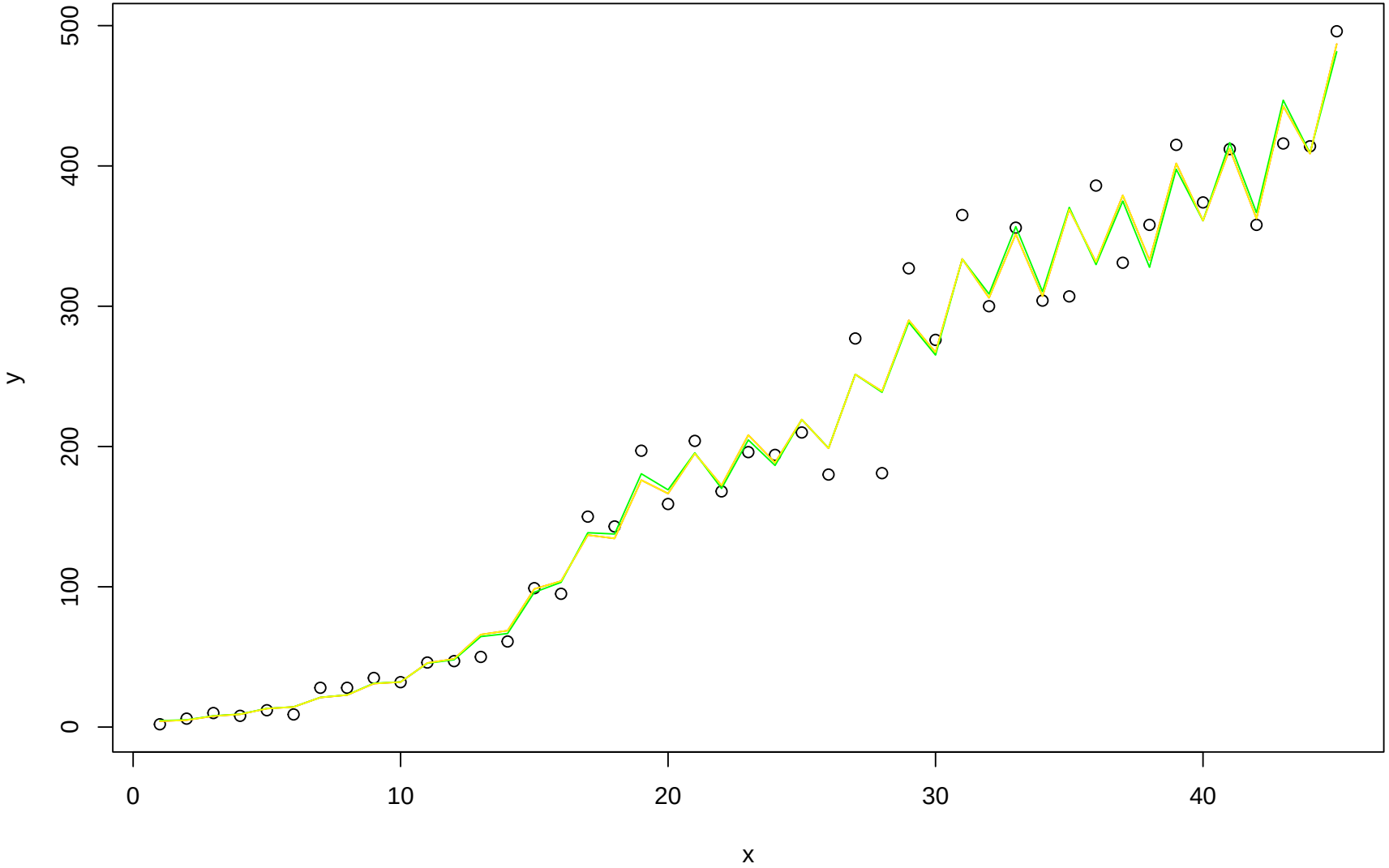
```
## GAMLSS-RS iteration 1: Global Deviance = 374.8622
## GAMLSS-RS iteration 2: Global Deviance = 374.8622
```

```r
aids2<-gamlss(y~pb(x,df=7)+qrt,data=aids,family=PO) # fix df's
```

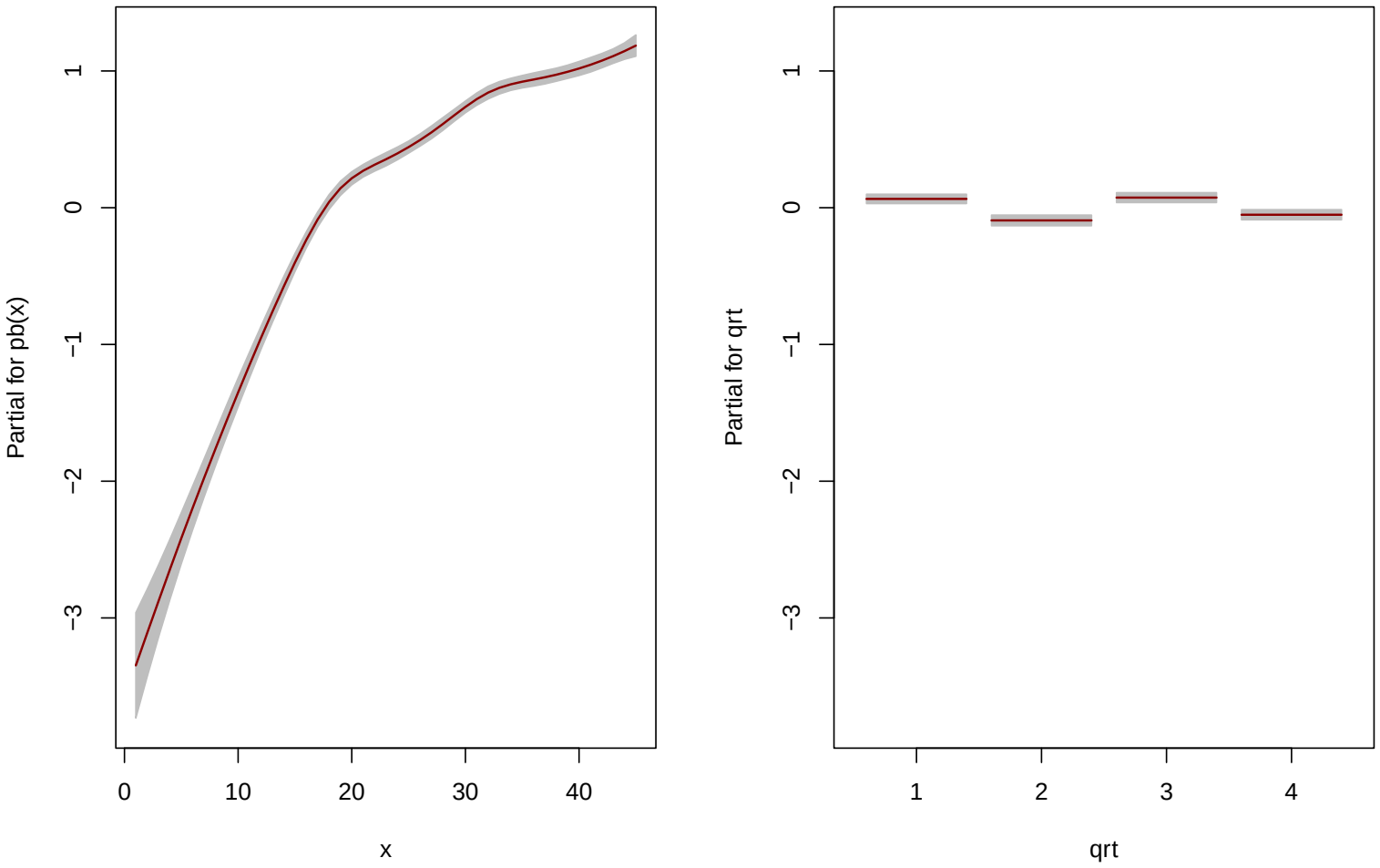
```
## GAMLSS-RS iteration 1: Global Deviance = 374.8366
## GAMLSS-RS iteration 2: Global Deviance = 374.8366
```

```r
aids3<-gamlss(y~pb(x)+qrt,data=aids,family=PO) # estimate lambda
```

```
## GAMLSS-RS iteration 1: Global Deviance = 387.1462
## GAMLSS-RS iteration 2: Global Deviance = 387.1547
## GAMLSS-RS iteration 3: Global Deviance = 387.1547
```


y
0
100
200
300
400
500
x
0
10
20
30
40

```r
par(mfrow=c(1,2))
term.plot(aids3)
```



Using ga() in library(gamlss.add)

```
aids4 <- gamlss(y~ga(~s(x)+qrt,method="REML"),data=aids,family=PO)
```

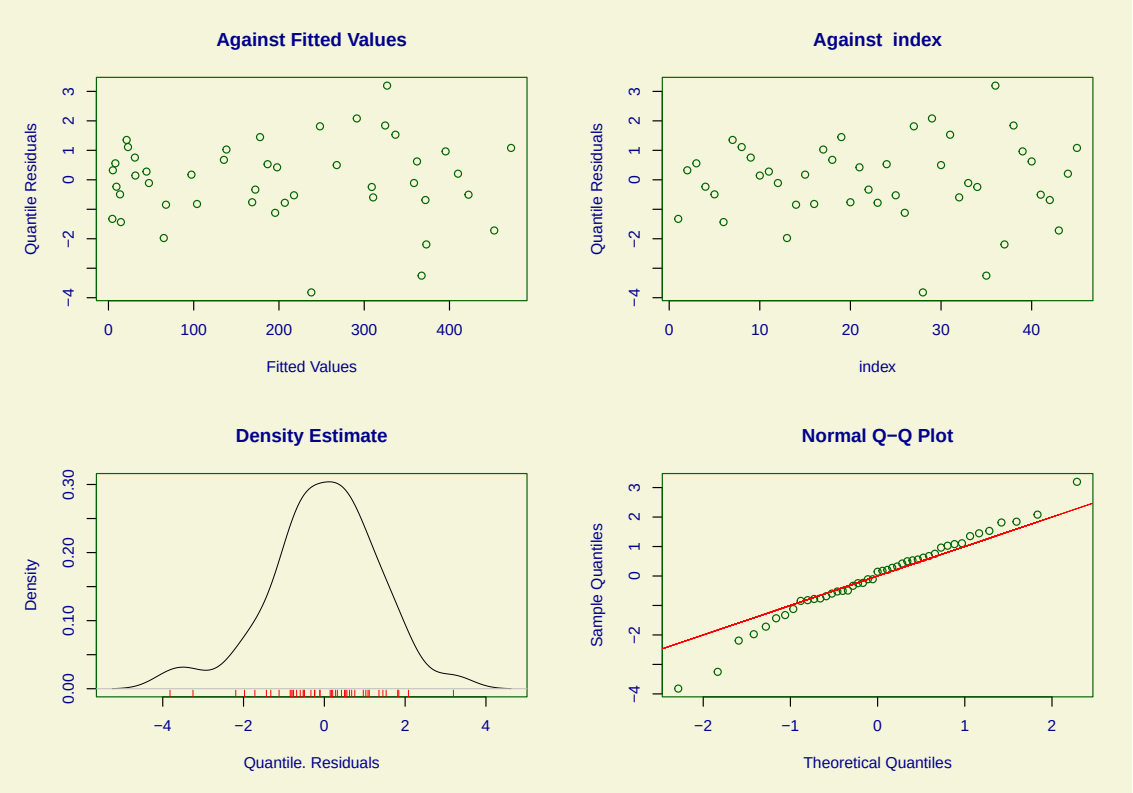
```
## GAMLSS-RS iteration 1: Global Deviance = 376.8816
## GAMLSS-RS iteration 2: Global Deviance = 376.8816
```

```r
summary(aids4)
```

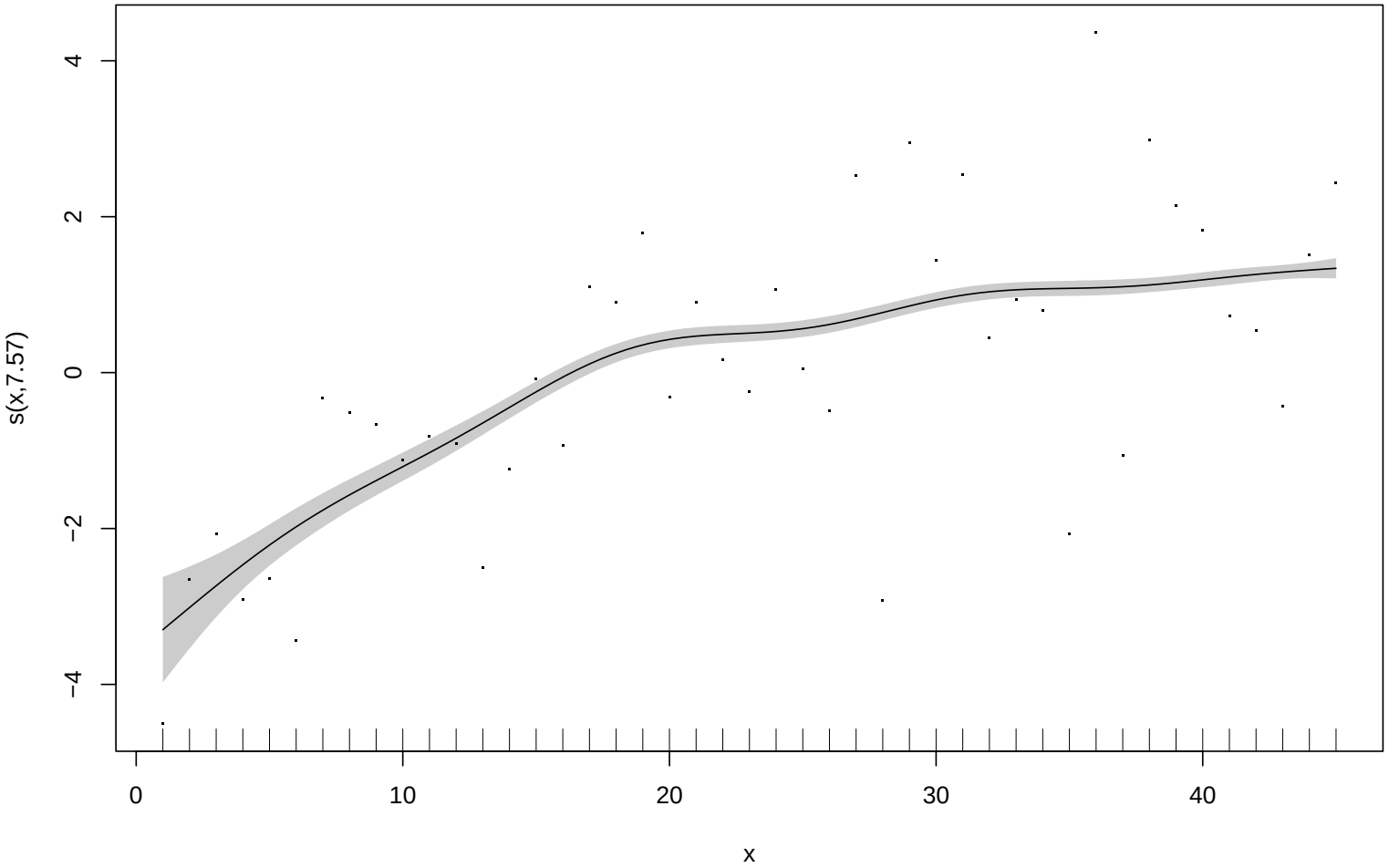
```
## *******************************************************************
## Family:  c("PO", "Poisson")
##
## Call:  gamlss(formula = y ~ ga(~s(x) + qrt, method = "REML"),
##     family = PO, data = aids)
##
## Fitting method: RS()
##
## -------------------------------------------------------------------
## Mu link function:  log
## Mu Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  5.29059    0.01053   502.5   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## -------------------------------------------------------------------
## NOTE: Additive smoothing terms exist in the formulas:
##  i) Std. Error for smoothers are for the linear effect only.
## ii) Std. Error for the linear terms maybe are not accurate.
## -------------------------------------------------------------------
## No. of observations in the fit:  45
## Degrees of Freedom for the fit:  11.57212
##       Residual Deg. of Freedom:  33.42788
##                       at cycle:  2
##
## Global Deviance:     376.8816
##             AIC:     400.0258
##             SBC:     420.9327
## *******************************************************************
```

```r
summary(getSmo(aids4))
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## Y.var ~ s(x) + qrt
##
## Parametric coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.471097   0.044363 -10.619 3.01e-12 ***
## qrt2        -0.161086   0.047577  -3.386  0.00183 **
## qrt3         0.005945   0.044491   0.134  0.89450
## qrt4        -0.118101   0.044957  -2.627  0.01291 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##        edf Ref.df     F p-value
## s(x) 7.572  8.352 145.7  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.959   Deviance explained = 97.4%
## GCV = 3.1961  Scale est. = 2.3742    n = 45
```

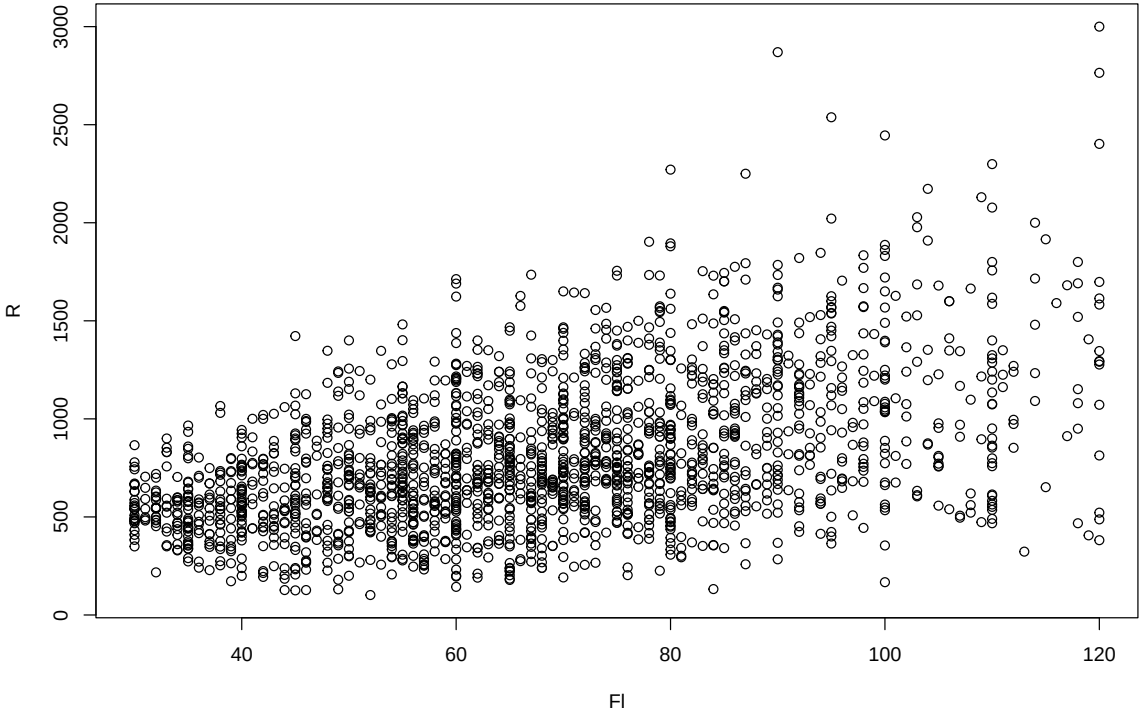



```
## ****************************************************************
##   Summary of the Randomised Quantile Residuals
##                          mean   =  -0.02754312
##                      variance   =  1.8385
##               coef. of skewness  =  -0.4222547
##               coef. of kurtosis  =  3.562297
## Filliben correlation coefficient  =  0.9879648
## ****************************************************************
```


s(x,7.57)
4
2
0
−2
−4
0
10
20
30
40
x

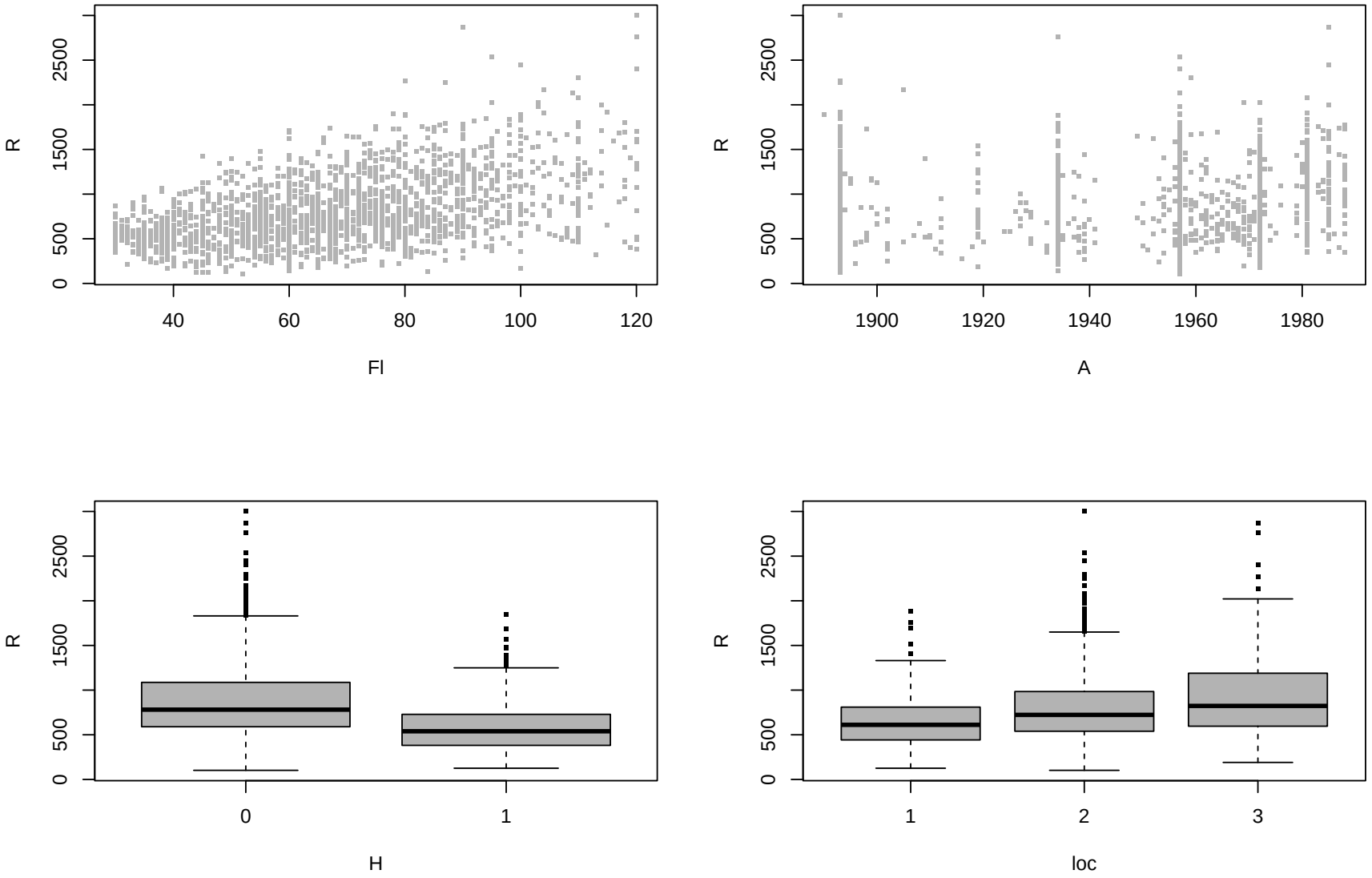
Example: Munich rent data

```
data(rent)
?rent
attach(rent)
plot(Fl,R)
```



A data frame with 1969 observations on the following 9 variables.

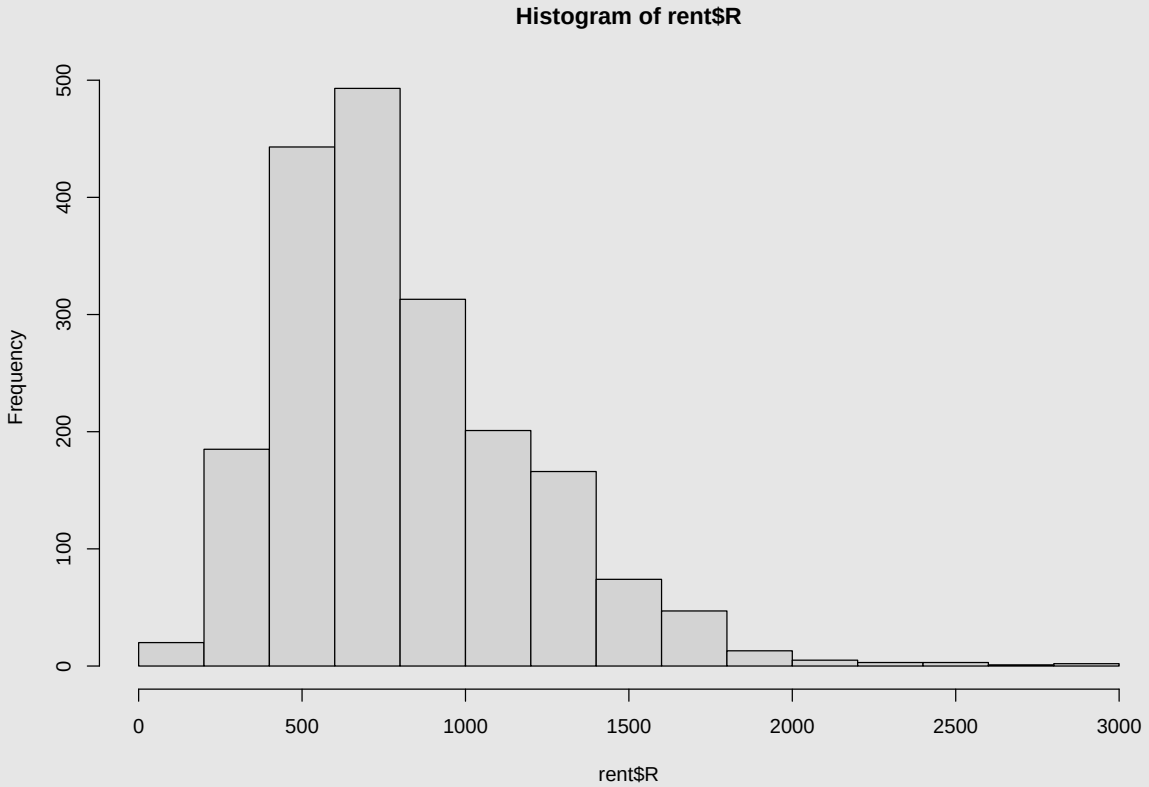
- `R`: rent response variable, the monthly net rent in DM, i.e. the monthly rent minus calculated or estimated utility cost
- `Fl`: floor space in square meters
- `A`: year of construction
- `Sp`: a variable indicating whether the location is above average, 1, (550 observations) or not, 0, (1419 observations)
- `Sm`: a variable indicating whether the location is below, 1, average (172 obs.) or not, 0, (1797 obs.)
- `B`: a factor with levels indicating whether there is a bathroom, 1, (1925 obs.) or not, 0, (44 obs.)
- `H`: a factor with levels indicating whether there is central heating, 1, (1580 obs.) or not, 0, (389 obs.)
- `L`: a factor with levels indicating whether the kitchen equipment is above average, 1, (161 obs.) or not, 0, (1808 obs.)
- `loc`: a factor (combination of Sp and Sm) indicating whether the location is below, 1, average, 2, or above average 3


R
0
500
1500
2500
40
60
80
100
120
Fl
R
0
500
1500
2500
1900
1920
1940
1960
1980
A
R
0
500
1500
2500
0
1
H
R
0
500
1500
2500
1
2
3
loc

Questions of interest

- Fit a `gamlss` model
- Which distribution fits well to the `rent` data?

```
hist(rent$R)
```

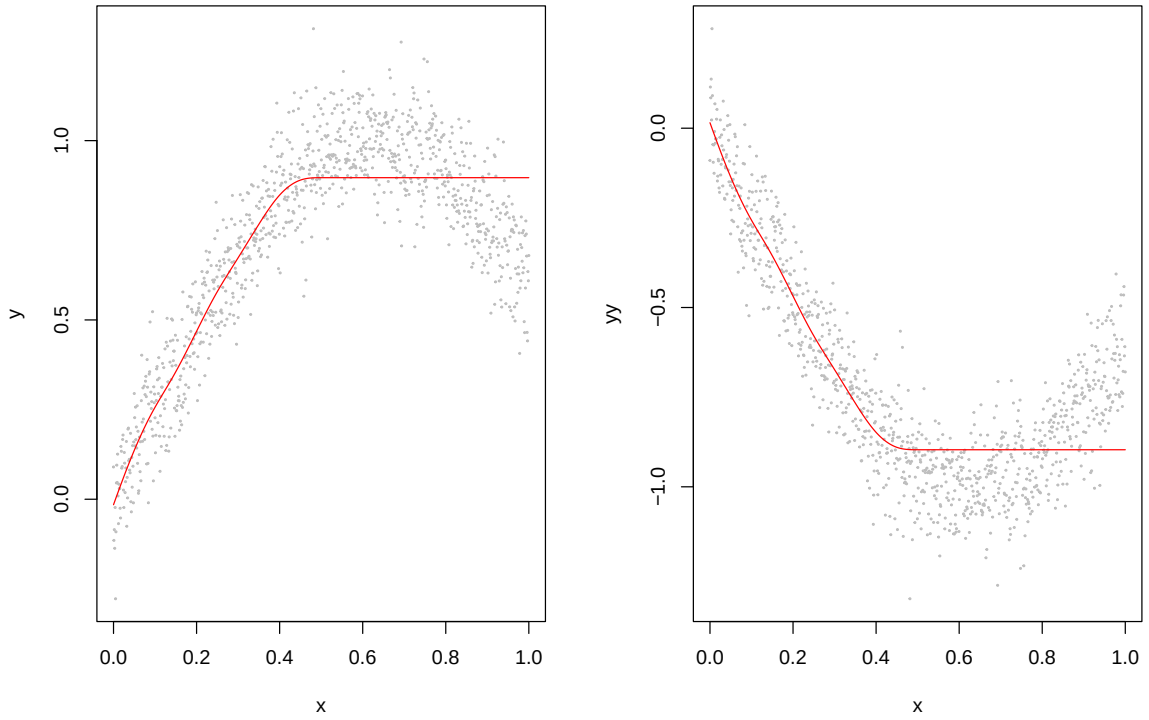


Tip fitDist function

```
distributions <- fitDist(
                    rent$R,
                    k = 2, # AIC
                    type = "realplus",
                    trace = FALSE,
                    try.gamlss = TRUE
                  )
distributions$fits
```

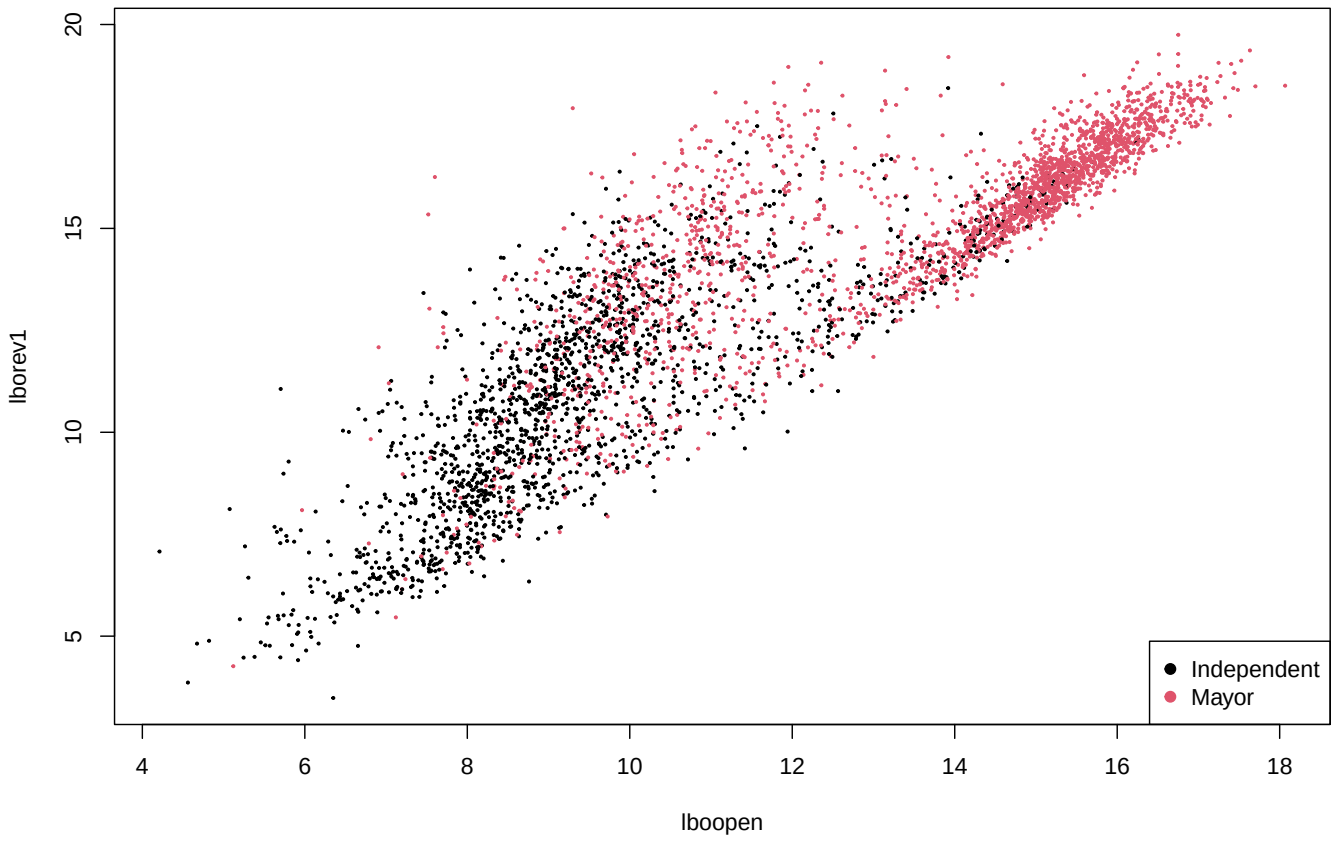
Monotonic smoothing (pbm())

```
set.seed(2022)
par(mfrow=c(1,2))
x = seq(0, 1, length = 1000)
p = 0.4
y = sin(2 * pi * p * x) + rnorm(1000) * 0.1
plot(y~x, cex=.2, col="gray")
m1 <- gamlss(y~pbm(x), trace=FALSE)
lines(fitted(m1)~x, col="red")
yy <- -y
plot(yy~x, cex=.2, col="gray")
m2 <- gamlss(yy~pbm(x, mono="down"), trace=FALSE)
lines(fitted(m2)~x, col="red")
```



Example: film90

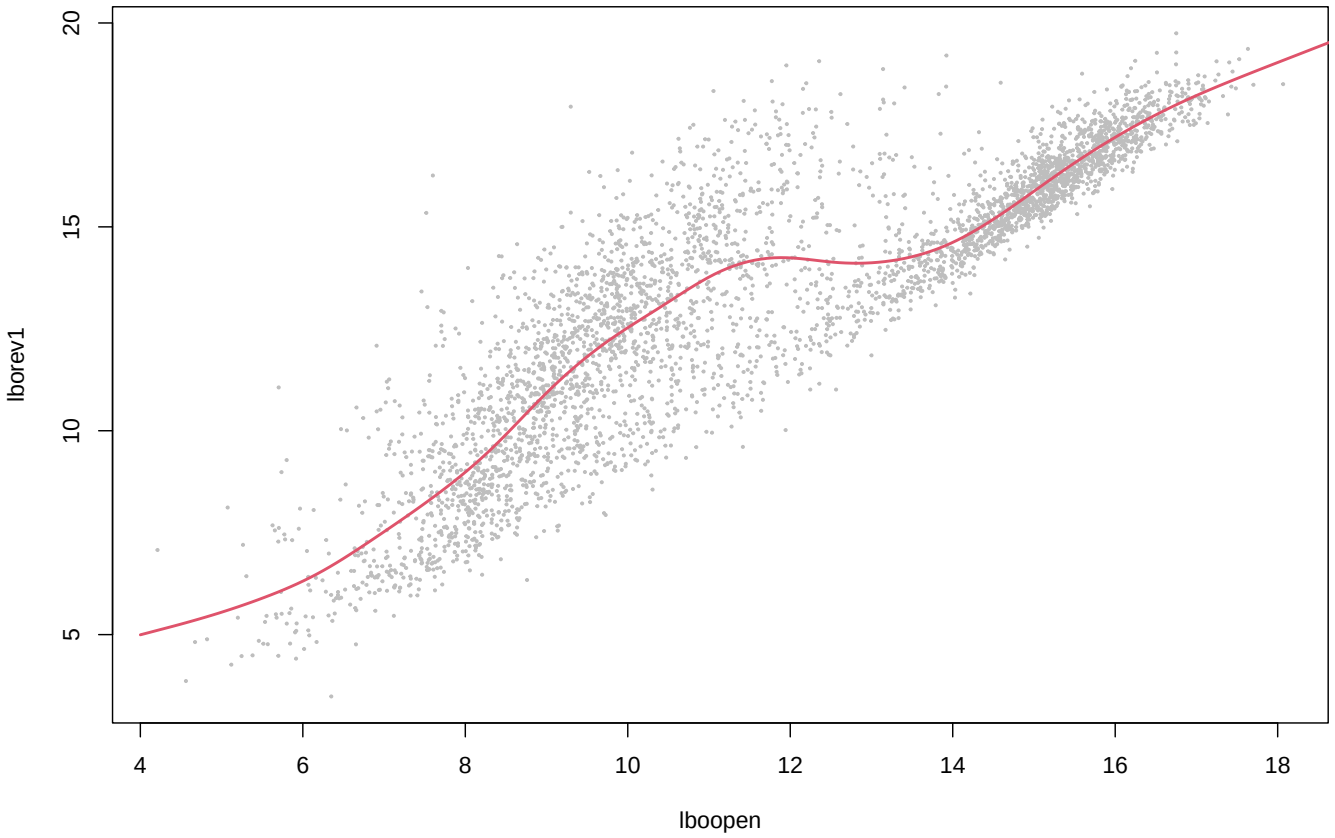
```
library(gamlss.data)
data(film90)
plot(lborev1~lboopen,pch=19,cex=0.25,col=as.numeric(dist),data=film90)
legend("bottomright",c("Independent","Mayor"),col = 1:2, pch=19,cex=1)
```



Fit a P-spline with `family=NO`

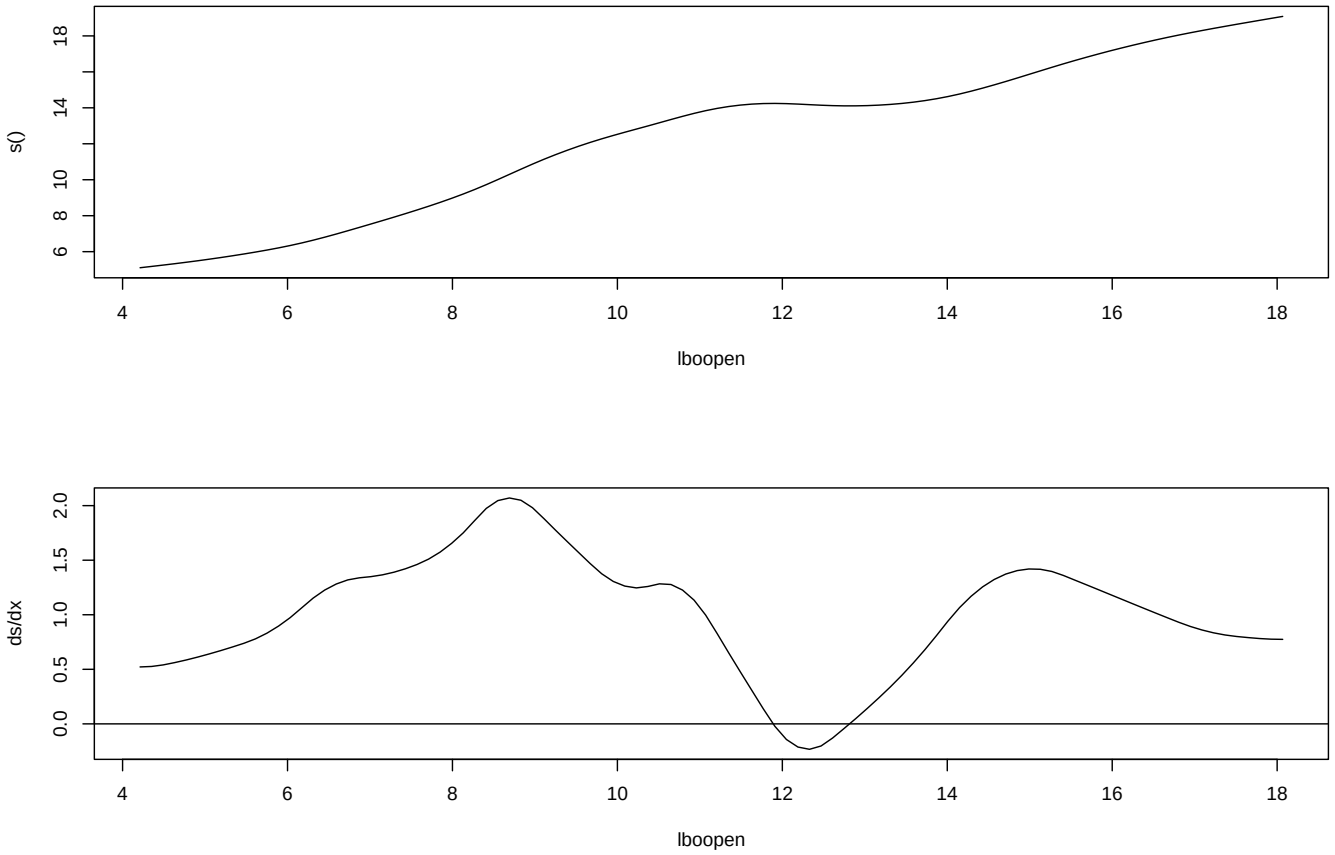
```
modelo_ps <- gamlss(
              formula = lborev1 ~ pb(lboopen),
              data    = film90,
              family  = NO,
              trace   = FALSE
            )
```

```
pred <- predict(modelo_ps,newdata = data.frame(lboopen = seq(4,20,l=200)))
plot(lborev1~lboopen,pch=19,col="grey",cex=0.25,data=film90)
lines(seq(4,20,l=200),pred,col=2,lwd=2)
```



Get partial effects with getPEF

```
getPEF(modelo_ps,term = "lboopen", parameter="mu",plot=TRUE)
```

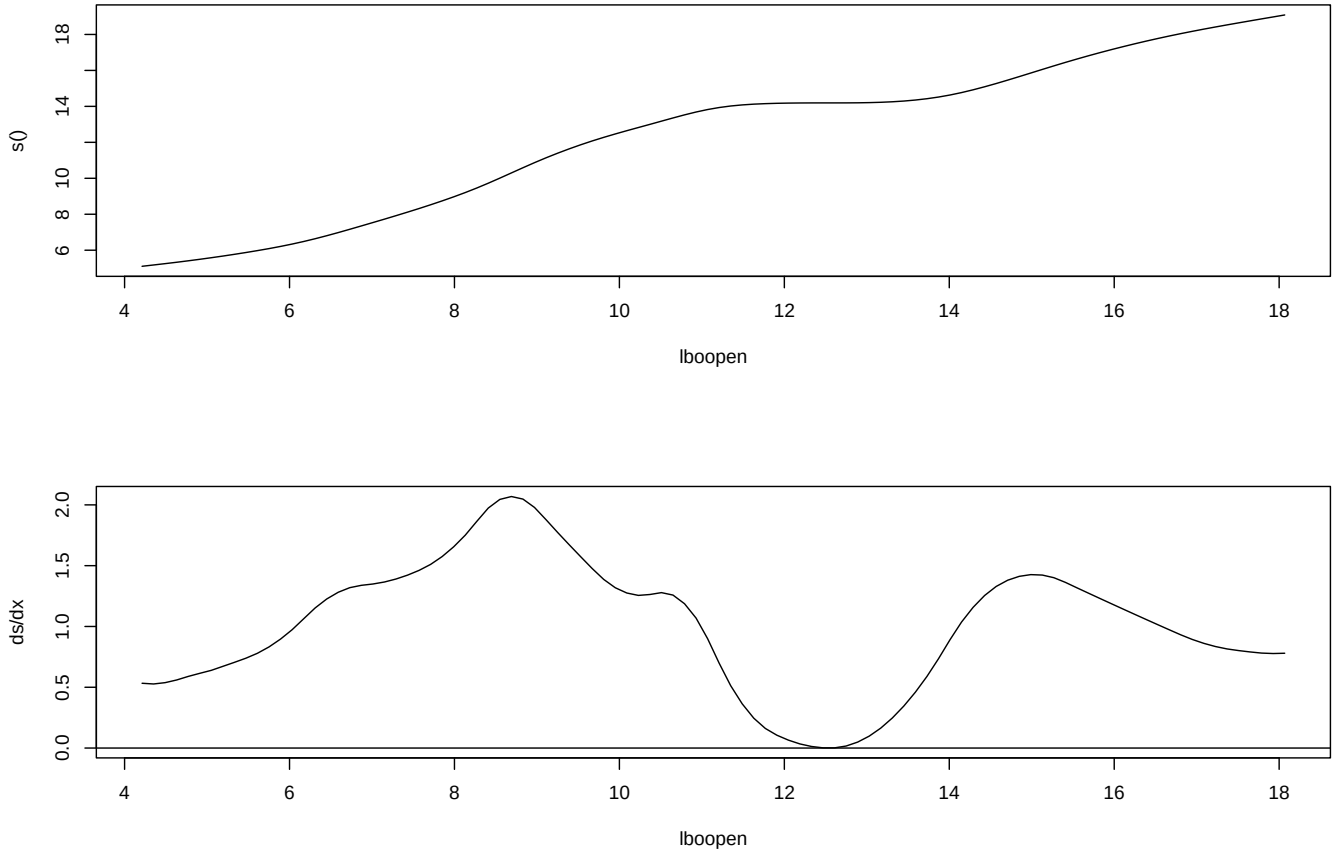


With `pbm(x,mono="up")`

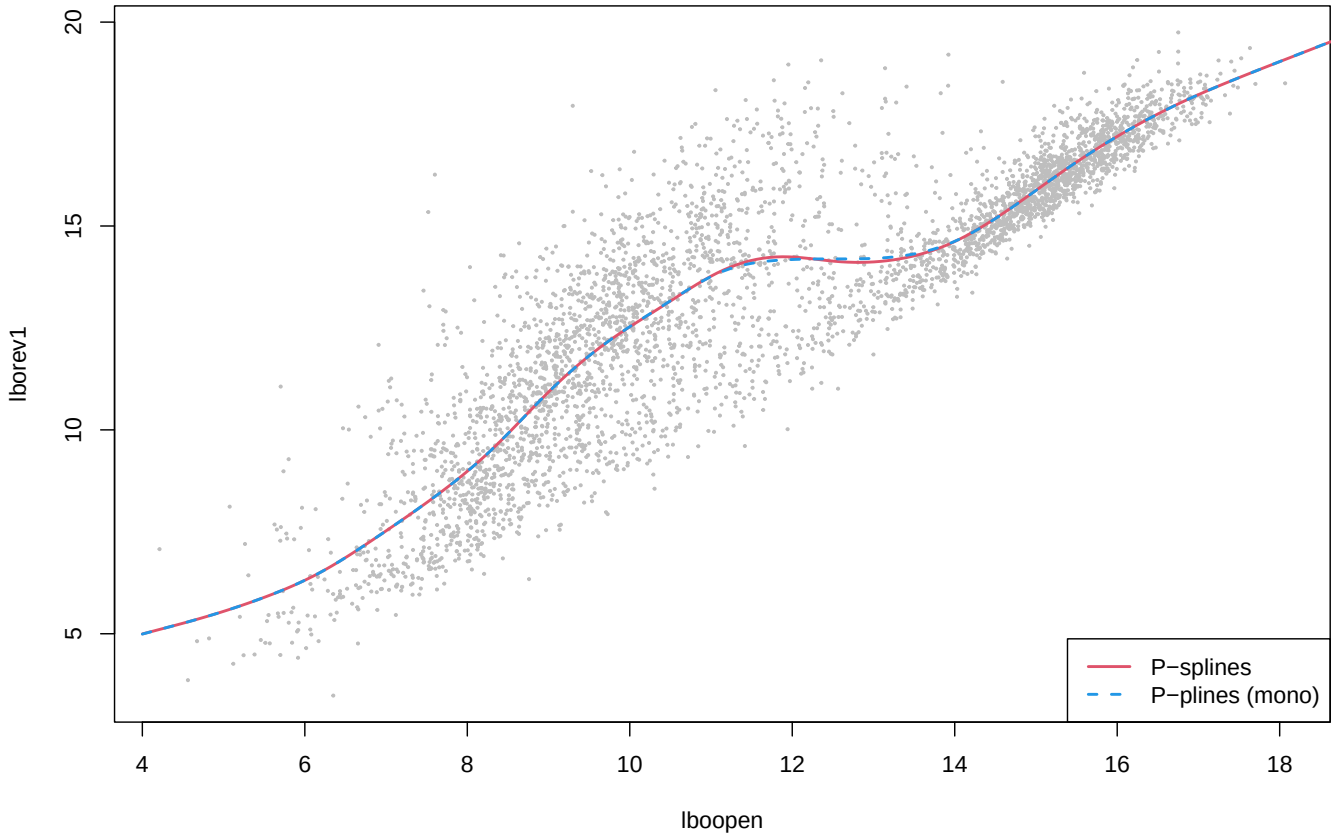
```
modelo_ps_mono <- gamlss(
                  formula = lborev1 ~ pbm(lboopen,mono="up"),
                  data    = film90,
                  family  = NO,
                  trace   = FALSE
                 )
```

Get partial effects

```
getPEF(modelo_ps_mono,term = "lboopen", parameter="mu",plot=TRUE)
```

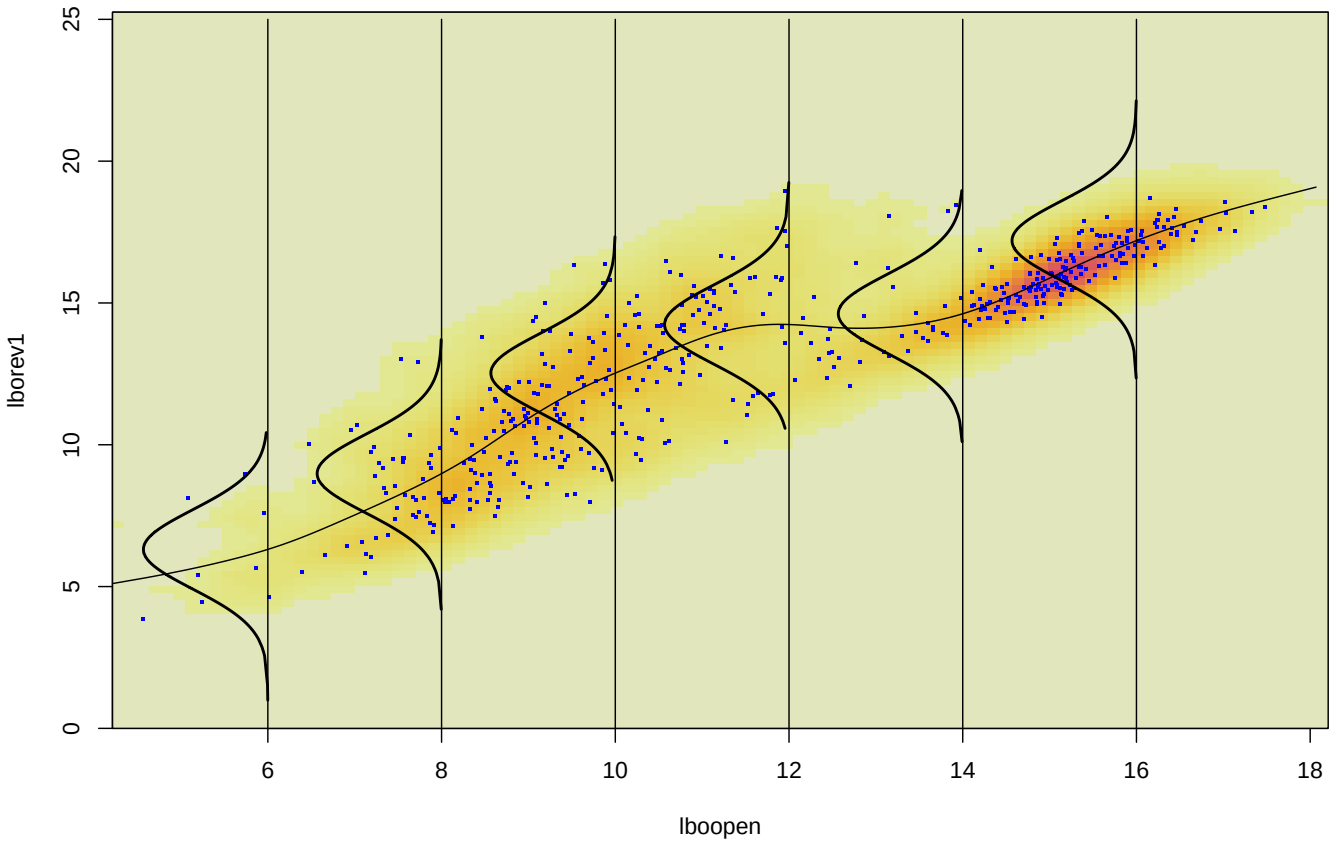


```
pred <- predict(modelo_ps,newdata = data.frame(lboopen = seq(4
pred_mono <- predict(modelo_ps_mono,newdata = data.frame(lboop
plot(lborev1~lboopen,pch=19,col="grey",cex=0.25,data=film90)
lines(seq(4,20,l=200),pred,col=2,lwd=2)
lines(seq(4,20,l=200),pred_mono,col=4,lwd=2,lty=2)
legend("bottomright",c("P-splines","P-plines (mono)"),col = c(
```



plotSimpleGamlss

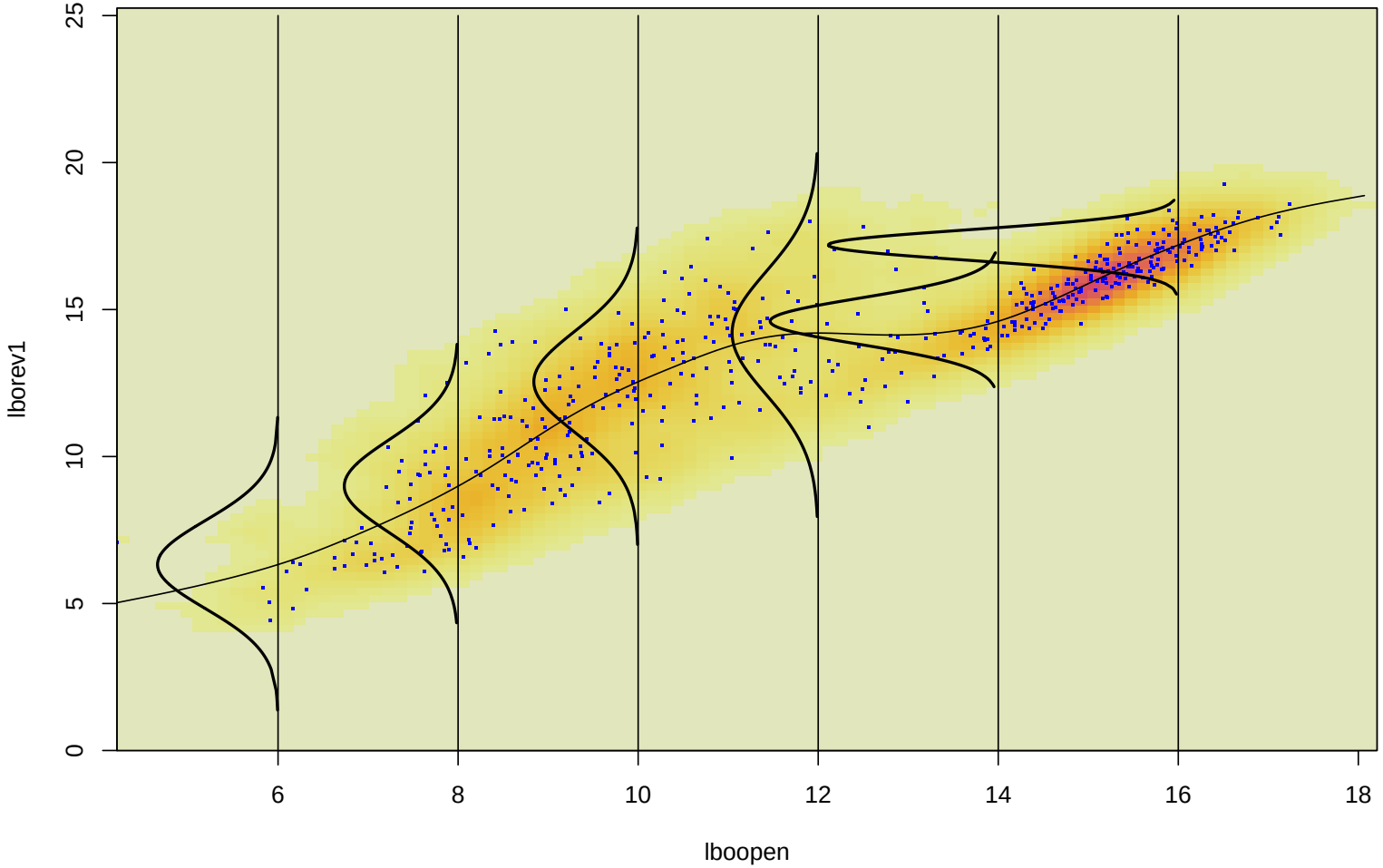
```
library(gamlss.util)
library(colorspace)

plotSimpleGamlss(
  y = lborev1, x = lboopen, model = modelo_ps, data  = film90,
  x.val = seq(6, 16, 2), val   = 5, N     =  1000, ylim  = c(0,25),
  cols  = heat_hcl(100)
)
```



Modelling the scale parameter with `sigma.formula`

```
modelo_gamlss <- gamlss(
                  formula = lborev1 ~ pb(lboopen),
                  sigma.formula = lborev1 ~ pb(lboopen),
                  data    = film90,
                  family  = NO,
                  trace   = FALSE
                )
```

gamlss plot



Inference with gamlss

- `vcov()`

```
vcov(modelo_ps, type="se")
vcov(modelo_gamlss, type="se")
```

- `rcov()`

```
rcov(modelo_ps, type="se")
rcov(modelo_gamlss, type="se")
```

confint()

```
confint(
  modelo_gamlss,
  what   = c("mu"),
  level  = 0.95,
  robust = TRUE
)
```

Variable selection in gamlss

rent data

```
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.1.2

##
## Attaching package: 'dplyr'

## The following object is masked from 'package:nlme':
##
##     collapse

## The following object is masked from 'package:MASS':
##
##     select

## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```r
# Modelos candidatos
modelo_1 <- gamlss(formula = precio ~ metros,
                   family = NO, data = datos, trace = FALSE)
modelo_2 <- gamlss(formula = precio ~ metros + anyo,
                   family = NO, data = datos, trace = FALSE)
modelo_3 <- gamlss(formula = precio ~ metros,
                   family = GA, data = datos, trace = FALSE)
modelo_4 <- gamlss(formula = precio ~ metros + anyo,
                   family = GA, data = datos, trace = FALSE)
```

Use function `GAIC` for model selection

```r
GAIC(modelo_1, modelo_2, modelo_3, modelo_4, k = 3)
```

```
##          df      AIC
## modelo_4  4 28000.20
## modelo_3  3 28107.43
## modelo_2  4 28355.14
## modelo_1  3 28472.35
```

gamlssCV

```
cv_modelo_1 <- gamlssCV(formula = precio ~ metros, family = NO, data = datos,
                        K.fold = 10, parallel = "multicore", ncpus = 4,
                        set.seed = 1)
cv_modelo_2 <- gamlssCV(formula = precio ~ metros + anyo, family = NO, data = datos,
                        K.fold = 10, parallel = "multicore", ncpus = 4,
                        set.seed = 1)
cv_modelo_3 <- gamlssCV(formula = precio ~ metros, family = GA, data = datos,
                        K.fold = 10, parallel = "multicore", ncpus = 4,
                        set.seed = 1)
cv_modelo_4 <- gamlssCV(formula = precio ~ metros + anyo, family = GA, data = datos,
                        K.fold = 10, parallel = "multicore", ncpus = 4,
                        set.seed = 1)

CV(cv_modelo_1, cv_modelo_2, cv_modelo_3, cv_modelo_4)
```

```
##             val[o.val]
## cv_modelo_4   27996.49
## cv_modelo_3   28104.19
## cv_modelo_2   28358.33
## cv_modelo_1   28476.79
```

stepGAICAll.A()

```
modelo_nulo <- gamlss(precio~1, data = datos, family = GA, trace = FALSE)
mejor_modelo <- stepGAICAll.A(
                  object = modelo_nulo,
                  scope  = list(lower=~1, upper=~metros+anyo+calefaccion+situacion)
                  k      = log(nrow(datos)),
                  parallel = "multicore",
                  ncpus  = 4
                )
```

`stepGAICAll.B()` follows a similar strategy as `stepGAICAll.A()`, but each term in the scope is fitted to all the parameters of the distribution.

Centile estimation

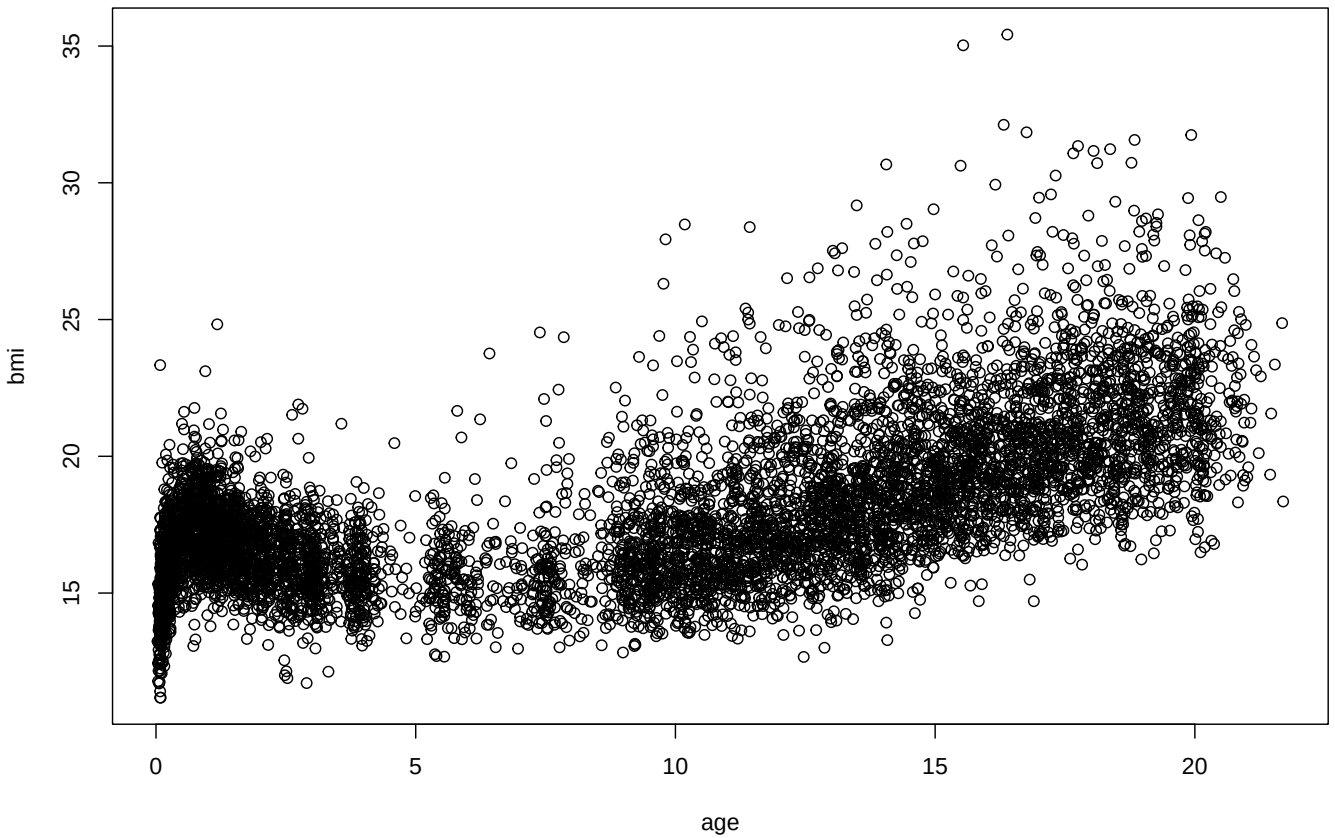
Centile estimation includes methods for estimating the age related distribution of human growth.

The standard estimation of centile curves involves two continuous variables:

```
* the response variable, that is, the variable we are interested in and for
* the explanatory variable age.
```

The Dutch Boys data

```
library(gamlss.data)
data(dbbmi)
plot(bmi~age, data=dbbmi)
```



`lms()` function

The `lms()` function is designated to implement the LMS (*lambda*, *mu*, *sigma*) method by Cole (1988)

GAMs for Quantile regression

Other R packages

`gamboostLSS`